

Mini-MapReduce Implementation

- Mini-MapReduce Implementation
 - Attribution
 - Summary
 - Quickstart - End-to-End Pipeline
 - * Pipeline Functionality
 - * Stopping the Demo
 - Components
 - * Virtual Environment
 - * `coordinator.proto`
 - * `Generator`
 - Usage
 - * `JobManager`
 - Jobs and Tasks
 - * `CoordinatorService`
 - * `Worker`
-

Attribution

Author

James Daniel Johnson

CWID

20183229

Course

CSIS 604 - Distributed Systems

Assignment

2.2 - MiniMapReduce

Summary

This submission encompasses a small-scale, problem-focused implementation of the **MapReduce** distributed architecture pattern. Due to its deterministic mapping (splitting and sorting) and reduction (merging) phases, the algorithm implemented here is **merge-sort**. While the core implementation is focused on the coordination of a master coordinator gRPC service and worker gRPC clients to distribute workloads, components that make up a job-to-task pipeline are included to demonstrate end-to-end distributed workload scenarios.

The coordinator service manage the allocation of tasks to the workers independently of bi-directional continuously-streaming connections which track the health of their connections, and also provides a daemon to reap tasks from stale connections. The clients use the bi-directional streaming connection (using the **Heartbeat** remote procedure call) and will try to reconnect in the event of an outage.

Quickstart - End-to-End Pipeline

A demonstration of the end-to-end functionality of the system has been included in the form of a shell script for convenience. The script starts up a generator

To run the entire pipeline at once, simply run the `map-reduce-demo.sh` in this directory

```
$ ./map-reduce-demo.sh
Starting generator in daemon mode...
Started generator with PID=58256
Generating 1000000 random integers at jobs/038604c0-f346-4c1d-806f-00fd83459347.json
Sleeping for 5 seconds before next drop...
Starting coordinator service...
Started coordinator service with PID=58281
DEBUG:CoordinatorService:Initializing CoordinatorService
INFO:JobManager:JobManager scanning directory at jobs...
DEBUG:CoordinatorService:Starting health daemon
INFO:CoordinatorService:Coordinator Initialized!
Server listening on port 50051
INFO:JobManager:Loading job from file jobs/038604c0-f346-4c1d-806f-00fd83459347.json (assign
INFO:JobManager:Loaded 1000000 integers. Slicing into chunks of 100000...
INFO:JobManager:Successfully created and queued 10 map tasks for job_id=038604c0-f346-4c1d-8
INFO:JobManager:Successfully removed job file 038604c0-f346-4c1d-806f-00fd83459347.json after
```

Pipeline Functionality The pipeline starts up a **Generator** daemon which will supply JSON-encoded data (lists of integers) to simulate a job intake queue. Next, the **CoordinatorService** starts up its internal **JobManager** to poll the `jobs/` directory for new jobs. The **JobManager** manages the lifetime of the jobs, including writing the results to the `results/` directory once the sort is complete

After the **CoordinatorService** is started, the `map-reduce-demo.sh` script creates 5 **Worker** clients that will be responsible for handling the tasks (in this case running a merge-sort).

Stopping the Demo To stop the demo script, simply apply Ctrl+C to send a signal interrupt

```

Sleeping for 5 seconds before next drop...
DEBUG:CoordinatorService:No task found for worker id=85a69385-4390-469d-b69c-b9dc765b9ff8.
INFO:Worker:No task assigned for worker id=85a69385-4390-469d-b69c-b9dc765b9ff8, idling before
^C
Commencing graceful shutdown
Commencing graceful shutdown...

Generator commencing graceful shutdown...
Commencing graceful shutdown

Commencing graceful shutdown

Commencing graceful shutdown
[!] Cleaning up background processes...

Commencing graceful shutdown
Sending SIGTERM to PIDs: 65768 65770 65785 65786 65787 65788 65789

Commencing graceful shutdown

Commencing graceful shutdown
Commencing graceful shutdown...

Commencing graceful shutdown

Commencing graceful shutdown

Commencing graceful shutdown
Worker with id=2b872fdd-378f-4114-91c2-0a42c5557f07 stopped
Worker with id=9dd67b6a-1373-46b4-9bc2-c25bf4f0fe24 stopped
Worker with id=85a69385-4390-469d-b69c-b9dc765b9ff8 stopped
Worker with id=2bc45ab5-c456-42a7-9863-c234658023e8 stopped
Worker with id=4a081775-093c-40c4-9c09-c1dcc8c13053 stopped

```

The results of the run will be in the `results/` directory

```

$ tree results
results
  26602677-1347-426d-872a-af439569db05_sorted.json
  5d20a7a9-33eb-4857-a2a7-f8d1c2c16feb_sorted.json
  6378fafa-176c-4f16-830c-61056f3d2a86_sorted.json
  e0215c21-9133-44fe-a8c6-8c196df427a8_sorted.json
  fe5670ab-8e24-48fb-a9e4-e00b4bdd56a9_sorted.json

```

```
1 directory, 5 files
```

Components

Virtual Environment

A virtual environment has been created to support dependency management within this service implementation. Prior to running any python commands, be sure to activate the environment

```
source .venv/bin/activate
```

coordinator.proto

The **protobuf** specification that outlines the remote procedure calls (RPC) and messages transmitted by means of those RPCs. The core of this file defines the interfaces of the RPCs themselves:

```
service CoordinatorService {  
    // Worker RPC's  
    rpc AssignTask (TaskRequest) returns (TaskAssignment);  
    rpc ReportTaskStatus (TaskStatusReport) returns (TaskStatusAck);  
    rpc Heartbeat (stream HeartbeatPing) returns (stream HeartbeatAck);  
}
```

To generate the python files necessary to support the service, run the following command with the virtual environment activated in the terminal:

```
python -m grpc_tools.protoc -Iprotos --python_out=. --pyi_out=. --grpc_python_out=. protos/
```

Generator

Generates a randomized list of integers and stores them into a file for consumption by the JobManager

Usage The Generator may be run on an ad-hoc basis by simply running the python script

```
$ python generator.py  
Generating 1000000 random integers at jobs/6378fafe-176c-4f16-830c-61056f3d2a86.json
```

For more information, use the `--help` or `-h` flags

```
$ python generator.py --help  
usage: generator.py [-h] [-o OUTPUT] [-M MAX] [-m MIN] [-n LENGTH] [-d] [-i INTERVAL]
```

Generator CLI for producing lists of integers for sorting

```

options:
  -h, --help            show this help message and exit
  -o, --output OUTPUT    Output path or directory. Defaults to jobs/<generated-uuid>.json
  -M, --max MAX          The maximum integer value. (Default=1000000)
  -m, --min MIN          The minimum integer value. (Default=-1000000)
  -n, --length LENGTH    The number of values to generate. (Default=1000000)
  -d, --daemon           Run as a daemon loop, dropping a new file periodically.
  -i, --interval INTERVAL
                        Interval in seconds between generation loops when running as a daemon

```

JobManager

The core function of the `JobManager` is to provide separation of concern by providing an abstraction layer between the `CoordinatorService`. It handles the lifetime management of `Jobs` and `Tasks` so that the `CoordinatorService` can remain focused on coordination and management of `Worker` health and coordination.

Jobs and Tasks In this parlance a `Job` can be thought of as an indeterminate series of `Tasks`, and in the context of `MapReduce` jobs can be divided into phases. Observe the code below

```

class TaskState(Enum):
    """
    The state of a task as it
    relates to assignment
    """

    UNASSIGNED = 0
    IN_PROGRESS = 1
    COMPLETED = 2

class JobPhase(Enum):
    """
    The phase of a job, determined
    by the collection of tasks
    """

    MAP = 0
    REDUCE = 1
    COMPLETE = 2

@dataclass

```

```

class Task:
    """
    The data-view representation of a Task
    """

    task_id: str
    job_id: str
    task_type: TaskType
    input: list[IntegerArray]
    results: list[int] | None
    state: TaskState = TaskState.UNASSIGNED

@dataclass
class Job:
    """
    The data-view representation of a Job
    """

    job_id: str
    job_result: list[int] | None
    num_items: int # Crucial to figuring out if job is completed
    phase: JobPhase = JobPhase.MAP
    map_results: Queue[list[int]] = field(default_factory=Queue)
    reduce_buffer: Queue[list[int]] = field(default_factory=Queue)

```

A key role in the management of **Job** lifetimes is the consumption of incoming jobs. The **JobManager** therefore necessarily runs a daemon thread to consume job files (usually created by the **Generator**) and use them to create an in-memory view of active **Jobs** and their child **Tasks**

CoordinatorService

The **CoordinatorService** can be thought of as the *brain* of the implementation as it carries the responsibilities with coordinating between **Worker** clients along different phases of various **Jobs** and **Tasks**, while simultaneously addressing worker health and task reallocation due to connection issues with the **Worker** clients.

Observe the daemon for this purpose:

```

def _worker_health_daemon(self):
    """
    Continuously runs in the background and monitors worker health.
    Removes stale worker instances and handles adding their assigned
    tasks back to the queue
    """

```

```

"""
self.logger.debug("Starting health daemon")
while not self.shutdown_event.is_set():
    now = time.time()

    with self.lock:
        for worker_id, info in list(self.workers.items()):
            if self.shutdown_event.is_set():
                break
            if info.is_active and (
                now - info.last_seen > self.heartbeat_timeout
            ):
                self.logger.warning(
                    "Timeout detected in worker id=%s: took more than %.2f seconds t
                    worker_id,
                    now - info.last_seen,
                )
                info.is_active = False
                if info.assignment is not None:
                    self.job_manager.revoke_task_assignment(info.assignment)
                    info.assignment = None
                self.workers.pop(worker_id, None)
                self.logger.warning("Worker id=%s removed", worker_id)
        if not self.shutdown_event.is_set():
            # sleep for 1 second
            self.shutdown_event.wait(1.0)

```

Supposing we have generated jobs in the jobs directory, we can simulate a worker crash or network fault.

```

$ python coordinator.py & sleep 5 && python worker.py > /dev/null 2>&1 & sleep 5 && kill %2
DEBUG:CoordinatorService:Initializing CoordinatorService
INFO:JobManager:JobManager scanning directory at jobs...
DEBUG:CoordinatorService:Starting health daemon
INFO:CoordinatorService:Coordinator Initialized!
Server listening on port 50051
INFO:JobManager:Created REDUCE task id=ffe4c3fc-4676-4b97-bfdc-df6ba1379fc7 for job id=f19d
INFO:JobManager:Assigned task id=40feff34-a520-4c5b-a3df-878fcaff3ca (type=REDUCE) of job i
INFO:JobManager:Assigned task id=37cd3bd6-5252-4924-8405-d5cd4bfc1b19 (type=REDUCE) of job i
INFO:JobManager:Created REDUCE task id=1371d877-e1cf-4e21-8451-1261533b4b29 for job id=f19d
INFO:JobManager:Assigned task id=e53b0144-31f6-4e1d-a5f8-e9013fb9a70a (type=REDUCE) of job i
INFO:JobManager:Assigned task id=38679980-d2a3-41db-a791-77227f79da20 (type=REDUCE) of job i
[2] + done      python worker.py > /dev/null 2>&1
ERROR:CoordinatorService:Error in heartbeat detection:
WARNING:CoordinatorService:Heartbeat stream closed for worker id=ad502eba-c97a-4d6a-877d-d10
WARNING:CoordinatorService:Timeout detected in worker id=ad502eba-c97a-4d6a-877d-d1d2ad21d0e

```

WARNING:CoordinatorService:Worker id=ad502eba-c97a-4d6a-877d-d1d2ad21d0eb removed

Worker

The workhorse of the implementation, the worker does the grunt-work of simple mapping and reduction tasks. To be associated with these small tasks, the workers maintain a very simple implementation devoid of the details of their assigned task. Nevertheless, they do have the initial responsibility to contact the coordinator, and as such they have a background daemon for this purpose

Observe the code below

```
def _heartbeat_daemon(self):
    """Continuous heartbeat daemon"""

    def heartbeat_generator() -> Iterator[HeartbeatPing]:
        while not self.shutdown_event.is_set():
            yield HeartbeatPing(worker_id=self.id, timestamp=int(time() * 1000))
            sleep(2.0)

    while not self.shutdown_event.is_set():
        try:
            self.logger.debug(
                "Attempting heartbeat ping with Coordinator, worker id=%s",
                self.id,
            )
            response_iterator: Iterator[HeartbeatAck] = self.stub.Heartbeat(
                heartbeat_generator()
            )

            for response in response_iterator:
                if not response.acknowledged:
                    self.logger.warning(
                        "Heartbeat not acknowledged by coordinator, worker id=%s",
                        self.id,
                    )
                else:
                    self.logger.debug(
                        "Heartbeat acknowledged for worker %s", self.id
                    )

        except RpcError as e:
            self.logger.error("Heartbeat Stream Disconnected: %s", e.details())

    if not self.shutdown_event.is_set():
        self.logger.info(
```

```

        "Worker id=%s waiting 2 seconds before attempting reconnection...",
        self.id,
    )
    self.shutdown_event.wait(2.0)

```

This heartbeat is resilient enough to allow them to reconnect even if the coordinator is not up

```

$ python worker.py & sleep 2 && python coordinator.py & sleep 3 && kill %1 %2
[1] 71205
INFO:Worker:Starting up heartbeat thread for worker id=375951be-a669-4294-8d24-b7111ebf3d52
DEBUG:Worker:Attempting heartbeat ping with Coordinator, worker id=375951be-a669-4294-8d24-b7111ebf3d52
INFO:Worker:Starting up task thread for worker id=375951be-a669-4294-8d24-b7111ebf3d52
INFO:Worker:Starting up task polling for worker id=375951be-a669-4294-8d24-b7111ebf3d52
ERROR:Worker:Disconnected from Coordinator Service: failed to connect to all addresses; last error:
ERROR:Worker:Heartbeat Stream Disconnected: failed to connect to all addresses; last error:
INFO:Worker:Worker id=375951be-a669-4294-8d24-b7111ebf3d52 waiting 2 seconds before attempting
[2] 71221
DEBUG:Worker:Attempting heartbeat ping with Coordinator, worker id=375951be-a669-4294-8d24-b7111ebf3d52
ERROR:Worker:Disconnected from Coordinator Service: failed to connect to all addresses; last error:
ERROR:Worker:Heartbeat Stream Disconnected: failed to connect to all addresses; last error:
INFO:Worker:Worker id=375951be-a669-4294-8d24-b7111ebf3d52 waiting 2 seconds before attempting
DEBUG:CoordinatorService:Initializing CoordinatorService
INFO:JobManager:JobManager scanning directory at jobs...
DEBUG:CoordinatorService:Starting health daemon
INFO:CoordinatorService:Coordinator Initialized!
Server listening on port 50051
DEBUG:Worker:Attempting heartbeat ping with Coordinator, worker id=375951be-a669-4294-8d24-b7111ebf3d52
DEBUG:CoordinatorService:No task found for worker id=375951be-a669-4294-8d24-b7111ebf3d52.
DEBUG:CoordinatorService:Received ping from worker id=375951be-a669-4294-8d24-b7111ebf3d52
INFO:Worker:No task assigned for worker id=375951be-a669-4294-8d24-b7111ebf3d52, idling before
INFO:CoordinatorService:Worker id=375951be-a669-4294-8d24-b7111ebf3d52 registered via heartbeat
DEBUG:CoordinatorService:Heartbeat acknowledged for worker with id=375951be-a669-4294-8d24-b7111ebf3d52
DEBUG:Worker:Heartbeat acknowledged for worker 375951be-a669-4294-8d24-b7111ebf3d52

Commencing graceful shutdown...
Commencing graceful shutdown
Worker with id=375951be-a669-4294-8d24-b7111ebf3d52 stopped

[1] - done      python worker.py
ERROR:CoordinatorService:Error in heartbeat detection:
WARNING:CoordinatorService:Heartbeat stream closed for worker id=375951be-a669-4294-8d24-b7111ebf3d52
[2] + done      python coordinator.py

```